JNTUH R22

COMPILER DESIGN (CD) LAB

8 C Programs with Sample Inputs and Outputs

Prepared for: External Practical Examination
Language: C

Includes symbol table, lexical analysis, type checking, stack allocation, recursive descent parsing and FIRST set.

1. Implementation of Symbol Table

A symbol table stores identifier details such as variable names and their data types.

C Program

```
#include <stdio.h>

struct Symbol {
    char name[20];
    char type[20];
};

int main() {
    struct Symbol s[20];
    int n, i;

    printf("Enter number of symbols: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter name and type: ");
        scanf("%19s %19s", s[i].name, s[i].type);
    }

    printf("\\nSymbol Table\\n");
    printf("Name\\tType\\n");

    for (i = 0; i < n; i++)
        printf("%s\\t%s\\n", s[i].name, s[i].type);

    return 0;
}
```

Sample Input

```
3
a int
b float
c char
```

Expected Output

```
Symbol Table
Name    Type
a       int
b       float
c       char
```

2. Lexical Analyzer to Recognize Basic Patterns

This simple character-level version identifies letters, digits, arithmetic operators and other symbols. It is intended for a short demonstration.

C Program

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char s[100];
    int i = 0;

    printf("Enter an expression: ");
    scanf("%99s", s);

    while (s[i] != '\\0') {
        if (isalpha((unsigned char)s[i]) || s[i] == '_')
            printf("%c - Identifier\\n", s[i]);
        else if (isdigit((unsigned char)s[i]))
            printf("%c - Constant\\n", s[i]);
        else if (s[i] == '+' || s[i] == '-' ||
                 s[i] == '*' || s[i] == '/')
            printf("%c - Operator\\n", s[i]);
        else
            printf("%c - Special Symbol\\n", s[i]);
        i++;
```

```
    }
    return 0;
}
```

Sample Input

```
a+b*5
```

Expected Output

```
a - Identifier
+ - Operator
b - Identifier
* - Operator
5 - Constant
```

Note: This compact version reads a single expression without spaces; it does not group multi-character identifiers or detect comments.

3. Implementation of Type Checking

This program checks two numeric operand types. If either operand is float, the result is float; otherwise the result is int. Other types are rejected.

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char t1[10], t2[10];

    printf("Enter first operand type: ");
    scanf("%9s", t1);
    printf("Enter second operand type: ");
    scanf("%9s", t2);

    if ((strcmp(t1, "int") == 0 || strcmp(t1, "float") == 0) &&
        (strcmp(t2, "int") == 0 || strcmp(t2, "float") == 0)) {
        if (strcmp(t1, "float") == 0 || strcmp(t2, "float") == 0)
            printf("Result type: float\\n");
        else
            printf("Result type: int\\n");
    } else {
        printf("Type Error\\n");
    }
    return 0;
}
```

Sample Input

```
int
float
```

Expected Output

```
Result type: float
```

4. Storage Allocation Strategy – Stack

This program demonstrates stack behaviour using an array. Stack follows LIFO (Last In, First Out).

C Program

```
#include <stdio.h>

int main() {
    int stack[20], top = -1, n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n < 1 || n > 20) {
        printf("Invalid number of elements\\n");
        return 0;
```

```
    }

    printf("Enter elements:\\n");
    for (i = 0; i < n; i++)
        scanf("%d", &stack[++top]);

    printf("Stack elements (pop order):\\n");
    while (top >= 0)
        printf("%d\\n", stack[top--]);

    return 0;
}
```

Sample Input

```
3
10
20
30
```

Expected Output

```
Stack elements (pop order):
30
20
10
```

5. C Program to Implement a Lexical Analyzer

This version groups complete identifiers and integer constants, recognizes common operators and reports a // comment. Enter a single-line statement.

C Program

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char s[200];
    int i = 0;

    printf("Enter a statement: ");
    fgets(s, sizeof(s), stdin);

    while (s[i] != '\\0') {
        if (isspace((unsigned char)s[i])) {
            i++;
        }
        else if (s[i] == '/' && s[i + 1] == '/') {
            printf("Comment\\n");
            break;
        }
        else if (isalpha((unsigned char)s[i]) || s[i] == '_') {
            printf("Identifier: ");
            while (isalnum((unsigned char)s[i]) || s[i] == '_')
                putchar(s[i++]);
            printf("\\n");
        }
        else if (isdigit((unsigned char)s[i])) {
            printf("Constant: ");
            while (isdigit((unsigned char)s[i]))
                putchar(s[i++]);
            printf("\\n");
        }
        else if (s[i] == '+' || s[i] == '-' || s[i] == '*' ||
                 s[i] == '/' || s[i] == '=') {
            printf("Operator: %c\\n", s[i++]);
        }
        else {
            printf("Special Symbol: %c\\n", s[i++]);
        }
    }
    return 0;
}
```

Sample Input

```
int a=10+b;
```

Expected Output

```
Identifier: int
Identifier: a
Operator: =
Constant: 10
Operator: +
Identifier: b
Special Symbol: ;
```

Note: This is a teaching-level lexer: it treats keywords as identifiers and does not implement strings, decimal constants, or multi-character operators.

6. Recursive Descent Parser – Expression Grammar

Original grammar: E -> E+T | T; T -> T*F | F; F -> (E) | id. Left recursion is removed before coding: E -> TE', E' -> +TE' | epsilon, T -> FT', T' -> *FT' | epsilon, F -> (E) | id. In this program, use i to represent id.

C Program

```
#include <stdio.h>
#include <stdlib.h>

char s[100];
int i = 0;

void E();
void T();
void F();

void error() {
    printf("Rejected\\n");
    exit(0);
}

void E() {
    T();
    while (s[i] == '+') {
        i++;
        T();
    }
}

void T() {
    F();
    while (s[i] == '*') {
        i++;
        F();
    }
}

void F() {
    if (s[i] == 'i') {
        i++;                    /* i represents id */
    } else if (s[i] == '(') {
        i++;
        E();
        if (s[i] == ')')
            i++;
        else
            error();
    } else {
        error();
    }
}

int main() {
    printf("Enter expression (use i for id): ");
    scanf("%99s", s);

    E();

    if (s[i] == '\\0')
        printf("Accepted\\n");
    else
        printf("Rejected\\n");
```

```
    return 0;
}
```

Sample Input

```
i+i*i
```

Expected Output

```
Accepted
```

7. Recursive Descent Parser – List Grammar

Original grammar: S -> (L) | a; L -> L,S | S. Remove left recursion: S -> (L) | a; L -> SL'; L' -> ,SL' | epsilon.

C Program

```
#include <stdio.h>
#include <stdlib.h>

char s[100];
int i = 0;

void S();
void L();

void error() {
    printf("Rejected\\n");
    exit(0);
}

void S() {
    if (s[i] == 'a') {
        i++;
    } else if (s[i] == '(') {
        i++;
        L();
        if (s[i] == ')')
            i++;
        else
            error();
    } else {
        error();
    }
}

void L() {
    S();
    while (s[i] == ',') {
        i++;
        S();
    }
}

int main() {
    printf("Enter string: ");
    scanf("%99s", s);

    S();

    if (s[i] == '\\0')
        printf("Accepted\\n");
    else
        printf("Rejected\\n");

    return 0;
}
```

Sample Input

```
(a,a)
```

Expected Output

```
Accepted
```

8. C Program to Calculate FIRST Function

For the given expression grammar, every non-terminal can begin with either '(' or id. This compact program displays the derived FIRST sets.

C Program

```
#include <stdio.h>

int main() {
    /*
       Grammar:
       E -> E+T | T
       T -> T*F | F
       F -> (E) | id

       FIRST(F) = { (, id }
       Since E -> T and T -> F:
       FIRST(T) = FIRST(F)
       FIRST(E) = FIRST(T)
    */

    printf("FIRST(E) = { ( , id }\\n");
    printf("FIRST(T) = { ( , id }\\n");
    printf("FIRST(F) = { ( , id }\\n");

    return 0;
}
```

Sample Input

```
(No input required)
```

Expected Output

```
FIRST(E) = { ( , id }
FIRST(T) = { ( , id }
FIRST(F) = { ( , id }
```

Note: This is a grammar-specific FIRST-set demonstration, not a general-purpose FIRST calculator for arbitrary grammars.

Quick Viva Questions and Answers

1. What is a compiler? A compiler translates a source program into target or machine code.

2. What is lexical analysis? It is the first compiler phase; it converts input characters into tokens.

3. What is a token? A token is a category such as identifier, keyword, operator or constant.

4. What is a lexeme? A lexeme is the actual character sequence that matches a token.

5. What is a symbol table? It stores information about identifiers, such as name, type and scope.

6. What is parsing? Parsing checks whether tokens follow the grammar of a language.

7. What is recursive descent parsing? A top-down parsing method implemented using recursive procedures.

8. Why remove left recursion? A directly left-recursive grammar can cause infinite recursion in a recursive descent parser.

9. What is FIRST? FIRST(X) is the set of terminals that can appear first in strings derived from X.

10. What is type checking? It checks whether operations use compatible data types.

11. What is stack allocation? Memory is managed in stack order; it follows LIFO.

12. What is LIFO? Last In, First Out.

Set-wise Question List (as supplied)

Set 1: 1, 9
Set 2: 2, 10
Set 3: 6, 11
Set 4: 7, 12

Note: The mapping above is reproduced from the question numbers provided. Confirm the exact question-to-experiment mapping with your lab faculty or question paper.